

Red Neuronal para Detección de Cáncer de Mama

Proyecto NNapp: Clasificación con TensorFlow y PyTorch

Michael Sneyder Rodríguez Peña

Ingeniería de Sistemas

10 de septiembre de 2026

Agenda de la presentación

- 1 Contexto y Motivación
- 2 Conceptos Básicos de IA
- 3 El Dataset
- 4 Arquitectura de la Red
- 5 Tecnologías Usadas
- 6 API REST, FastAPI y Streamlit
- 7 Cómo Funciona la Aplicación
- 8 Cómo Ejecutarlo
- 9 Conclusiones

¿Cuál es el problema?

Situación real

El **cáncer de mama** es una de las enfermedades más comunes. Los médicos analizan muestras de tejido (biopsias) para saber si un tumor es **benigno** (no peligroso) o **maligno** (peligroso).

¿Cuál es el problema?

Situación real

El **cáncer de mama** es una de las enfermedades más comunes. Los médicos analizan muestras de tejido (biopsias) para saber si un tumor es **benigno** (no peligroso) o **maligno** (peligroso).

El reto

Analizar cada muestra a mano es lento, costoso y puede tener errores. **¿Podemos hacer que una computadora ayude a clasificar automáticamente cada muestra?**

¿Cuál es el problema?

Situación real

El **cáncer de mama** es una de las enfermedades más comunes. Los médicos analizan muestras de tejido (biopsias) para saber si un tumor es **benigno** (no peligroso) o **maligno** (peligroso).

El reto

Analizar cada muestra a mano es lento, costoso y puede tener errores. **¿Podemos hacer que una computadora ayude a clasificar automáticamente cada muestra?**

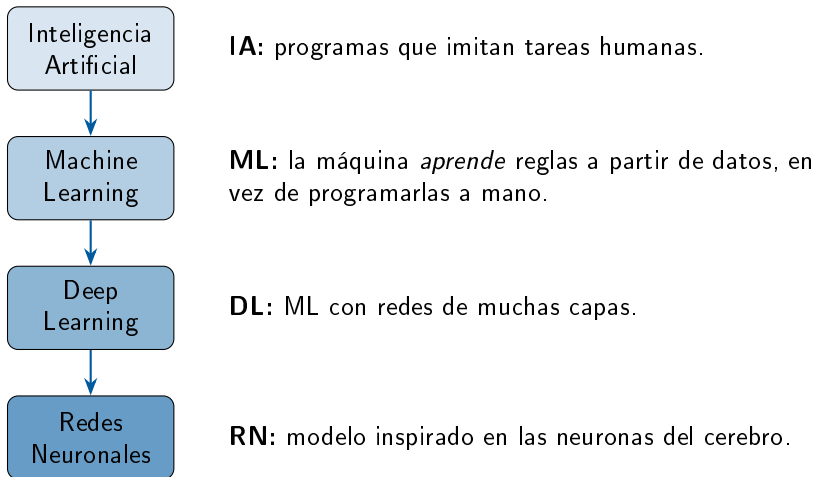
Nuestra solución

Construir una **Red Neuronal** que, a partir de 30 mediciones de una muestra, diga si el tumor es benigno o maligno.

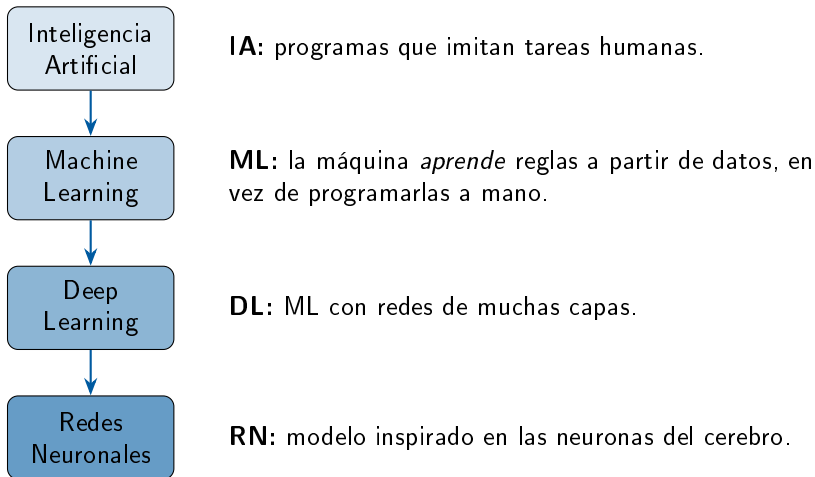
Objetivos del proyecto

- Entender **qué es una red neuronal** y cómo aprende.
- Crear una **aplicación web** completa que:
 - Entrene un modelo desde un archivo CSV.
 - Haga predicciones sobre datos nuevos.
- Comparar dos **frameworks** de Inteligencia Artificial: **TensorFlow** y **PyTorch**.
- Poner en práctica conceptos de Ingeniería de Sistemas: backend, frontend, API y arquitectura de software.

Inteligencia Artificial, Machine Learning y Deep Learning



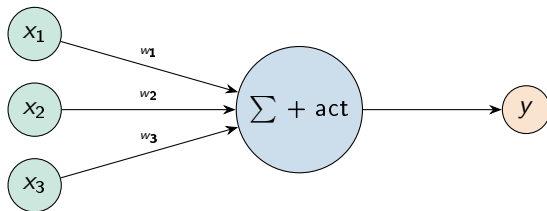
Inteligencia Artificial, Machine Learning y Deep Learning



Este proyecto usa **Deep Learning** (redes neuronales con varias capas).

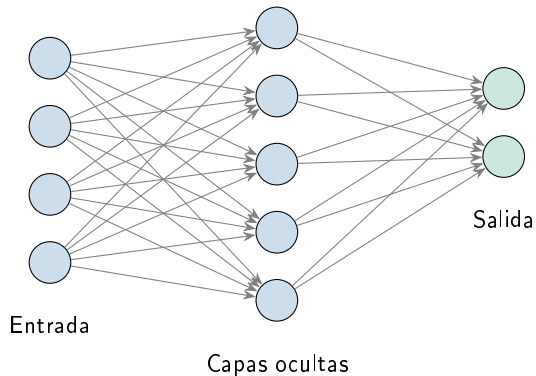
¿Qué es una neurona artificial?

- Recibe varias **entradas** ($x_1, x_2, \dots$).
- Cada entrada tiene un **peso** (w) que indica su importancia.
- Suma todo y le aplica una **función de activación**.
- Produce una **salida** (y).



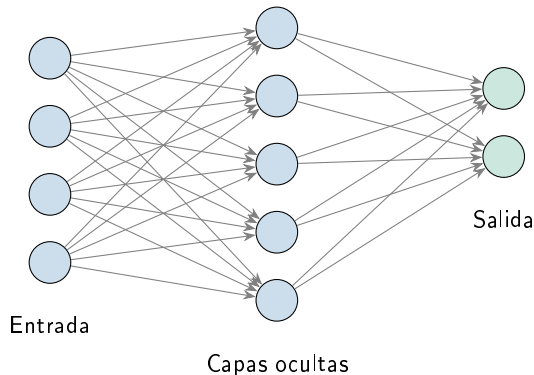
¿Qué es una red neuronal?

- Muchas neuronas conectadas en **capas**.



¿Qué es una red neuronal?

- Muchas neuronas conectadas en **capas**.



- Los datos entran por la izquierda y fluyen hacia la derecha.
- “Aprender” significa **ajustar los pesos** para acertar.

Clasificación binaria (lo que hacemos aquí)

¿Qué es clasificar?

Asignar una **categoría** (clase) a cada dato de entrada.

Clasificación binaria (lo que hacemos aquí)

¿Qué es clasificar?

Asignar una **categoría** (clase) a cada dato de entrada.

En este proyecto hay solo **dos categorías posibles**:

Clase	Significado
0	Maligno (tumor peligroso)
1	Benigno (tumor no peligroso)

Clasificación binaria (lo que hacemos aquí)

¿Qué es clasificar?

Asignar una **categoría** (clase) a cada dato de entrada.

En este proyecto hay solo **dos categorías posibles**:

Clase	Significado
0	Maligno (tumor peligroso)
1	Benigno (tumor no peligroso)

Ejemplo

Entrada: 30 mediciones de una biopsia → Salida: probabilidad de ser maligno. Si pasa de 50 %, se clasifica como maligno.

Dataset: Breast Cancer Wisconsin

- Conjunto de datos real y público, incluido en **Scikit-Learn**.
- Describe características de biopsias de tumores de mama.

Propiedad	Valor
Total de registros	569
Variables de entrada	30
Variable objetivo	target
Tipo de problema	Clasificación binaria
Clases	0 = maligno, 1 = benigno

Dataset: Breast Cancer Wisconsin

- Conjunto de datos real y público, incluido en **Scikit-Learn**.
- Describe características de biopsias de tumores de mama.

Propiedad	Valor
Total de registros	569
Variables de entrada	30
Variable objetivo	target
Tipo de problema	Clasificación binaria
Clases	0 = maligno, 1 = benigno

¿Qué son las 30 variables?

Mediciones numéricas del tumor: radio, textura, perímetro, área, simetría, etc. Cada una describe un aspecto de la célula.

¿Cómo se organizan los datos? (CSV)

- Los datos se guardan en un archivo **CSV** (texto con columnas).
- Cada **fila** = una muestra (una paciente/biopsia).
- Cada **columna** = una característica (feature).

mean radius	mean texture	...	mean area	target
17.99	10.38	...	1001.0	0
20.57	17.77	...	1326.0	0
13.54	14.36	...	545.8	1

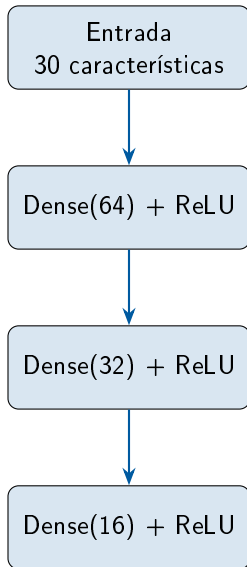
¿Cómo se organizan los datos? (CSV)

- Los datos se guardan en un archivo **CSV** (texto con columnas).
- Cada **fila** = una muestra (una paciente/biopsia).
- Cada **columna** = una característica (feature).

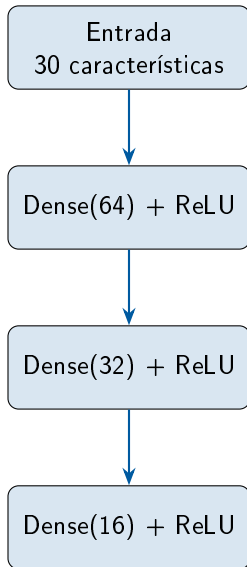
mean radius	mean texture	...	mean area	target
17.99	10.38	...	1001.0	0
20.57	17.77	...	1326.0	0
13.54	14.36	...	545.8	1

- Las columnas de la izquierda son las **entradas** (X).
- La columna target es la **respuesta correcta** (y).

Arquitectura de nuestra red neuronal



Arquitectura de nuestra red neuronal



ReLU (capas ocultas)

$$f(x) = \max(0, x)$$

- Si el valor es negativo $\rightarrow 0$.
- Si es positivo $\rightarrow$ lo deja pasar.
- Ayuda a la red a aprender patrones no lineales.

Sigmoid (capa de salida)

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Convierte cualquier número a un valor entre 0 y 1.
- Ese valor se interpreta como **probabilidad**.
- $> 0,5 \rightarrow$ benigno, $\leq 0,5 \rightarrow$ maligno.

Función de pérdida y optimizador

Función de pérdida (loss)

Binary Cross-Entropy

- Mide **qué tan mal** predice el modelo.
- Compara la predicción con la respuesta real.
- Meta: que la pérdida sea **lo más baja posible**.

Optimizador: Adam

- Es el “motor” que **ajusta los pesos**.
- Usa la pérdida para saber en qué dirección mejorar.
- Adam es rápido y muy usado en la práctica.

Función de pérdida y optimizador

Función de pérdida (loss)

Binary Cross-Entropy

- Mide **qué tan mal** predice el modelo.
- Compara la predicción con la respuesta real.
- Meta: que la pérdida sea **lo más baja posible**.

Optimizador: Adam

- Es el “motor” que **ajusta los pesos**.
- Usa la pérdida para saber en qué dirección mejorar.
- Adam es rápido y muy usado en la práctica.

Analogía: la pérdida es la “nota del examen” y el optimizador es el “estudio” que te hace mejorar en el siguiente intento.

¿Cómo aprende la red? (Entrenamiento)

- 1 **Adelante (forward):** los datos pasan por la red y sale una predicción.
- 2 **Calcular pérdida:** comparar predicción vs. respuesta real.
- 3 **Atrás (backward):** calcular cómo ajustar cada peso.
- 4 **Actualizar:** el optimizador modifica los pesos.
- 5 **Repetir** muchas veces (épocas).

¿Cómo aprende la red? (Entrenamiento)

- 1 **Adelante (forward)**: los datos pasan por la red y sale una predicción.
- 2 **Calcular pérdida**: comparar predicción vs. respuesta real.
- 3 **Atrás (backward)**: calcular cómo ajustar cada peso.
- 4 **Actualizar**: el optimizador modifica los pesos.
- 5 **Repetir** muchas veces (épocas).

Épocas (epochs)

Una **época** = ver **todas** las muestras una vez. Entrenamos 20 épocas: la red revisa el dataset completo 20 veces.

Dividir los datos: entrenar y probar

- No podemos evaluar con los mismos datos de entrenamiento (haría “trampa”).
- Dividimos el dataset:

Conjunto	Porcentaje	Uso
Entrenamiento	80 %	Aprender los pesos
Prueba (test)	20 %	Medir qué tan bien generaliza

Dividir los datos: entrenar y probar

- No podemos evaluar con los mismos datos de entrenamiento (haría “trampa”).
- Dividimos el dataset:

Conjunto	Porcentaje	Uso
Entrenamiento	80 %	Aprender los pesos
Prueba (test)	20 %	Medir qué tan bien generaliza

¿Por qué es importante?

Queremos que la red aprenda **patrones**, no que memorice. Si acierta en datos que nunca vio, entonces aprendió de verdad.

TensorFlow vs. PyTorch

TensorFlow (Keras)

- De Google.
- API **Keras**: simple y de alto nivel.
- Ideal para prototipos rápidos.
- Compila el modelo con `.compile()` y `.fit()`.

PyTorch

- De Meta (Facebook).
- Estilo más “Python” y flexible.
- Tú controlas el bucle de entrenamiento.
- Muy usado en investigación.

TensorFlow vs. PyTorch

TensorFlow (Keras)

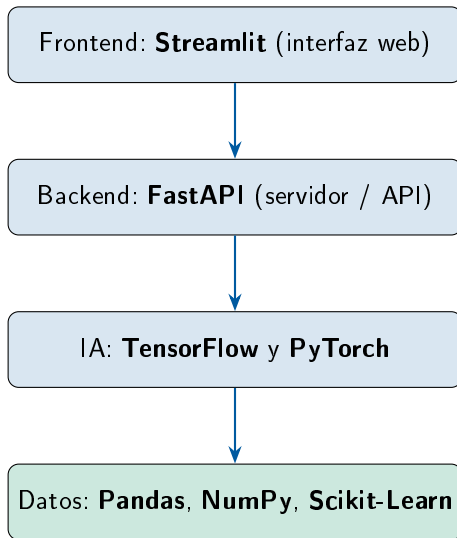
- De Google.
- API **Keras**: simple y de alto nivel.
- Ideal para prototipos rápidos.
- Compila el modelo con `.compile()` y `.fit()`.

PyTorch

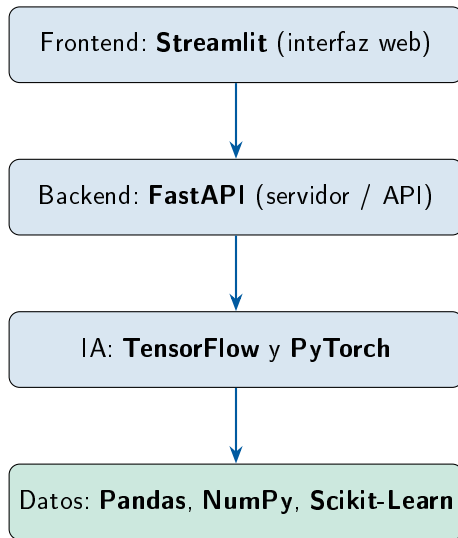
- De Meta (Facebook).
- Estilo más “Python” y flexible.
- Tú controlas el bucle de entrenamiento.
- Muy usado en investigación.

Ambos hacen lo mismo, pero con distinta forma de escribir el código. Nuestro proyecto soporta los dos.

El stack completo del proyecto



El stack completo del proyecto



¿Qué es una API?

Definición

API = *Application Programming Interface* (Interfaz de Programación de Aplicaciones). Es un **contrato** que permite que dos programas se comuniquen entre sí.

¿Qué es una API?

Definición

API = *Application Programming Interface* (Interfaz de Programación de Aplicaciones). Es un **contrato** que permite que dos programas se comuniquen entre sí.

Analogía: el mesero

Tú (cliente) pides un plato al **mesero** (API). El mesero lleva el pedido a la **cocina** (servidor) y te trae la comida (respuesta). No necesitas entrar a la cocina.

En nuestro proyecto

El **frontend** (Streamlit) y el **backend** (FastAPI) son programas distintos que se comunican a través de una API.

¿Qué es una API REST?

REST

REST = *Representational State Transfer*. Es un **estilo de arquitectura** para diseñar APIs web.

¿Qué es una API REST?

REST

REST = *Representational State Transfer*. Es un **estilo de arquitectura** para diseñar APIs web.

- Se comunica usando **HTTP** (el mismo protocolo de las páginas web).
- Usa los **métodos HTTP** para indicar la acción:
 - GET: leer información.
 - POST: crear / enviar datos.
 - PUT: actualizar.
 - DELETE: borrar.
- Los datos viajan normalmente en formato **JSON**.

¿Qué es una API REST?

REST

REST = *Representational State Transfer*. Es un **estilo de arquitectura** para diseñar APIs web.

- Se comunica usando **HTTP** (el mismo protocolo de las páginas web).
- Usa los **métodos HTTP** para indicar la acción:
 - GET: leer información.
 - POST: crear / enviar datos.
 - PUT: actualizar.
 - DELETE: borrar.
- Los datos viajan normalmente en formato **JSON**.

Nuestra app usa POST porque enviamos archivos CSV al servidor.

¿Qué es un endpoint?

Definición

Un **endpoint** es una **URL específica** que expone una funcionalidad del backend. Cada endpoint hace **una tarea**.

¿Qué es un endpoint?

Definición

Un **endpoint** es una **URL específica** que expone una funcionalidad del backend. Cada endpoint hace **una tarea**.

Método	Endpoint	Qué hace
POST	/train	Entrena el modelo con un CSV
POST	/predict/	Predice sobre datos nuevos

¿Qué es un endpoint?

Definición

Un **endpoint** es una **URL específica** que expone una funcionalidad del backend. Cada endpoint hace **una tarea**.

Método	Endpoint	Qué hace
POST	/train	Entrena el modelo con un CSV
POST	/predict/	Predice sobre datos nuevos

Ejemplo

`http://localhost:8000/train` → sube el CSV y entrena.

`http://localhost:8000/predict/` → sube datos y predice.

¿Qué es FastAPI?

Es un **framework de Python** para crear APIs web de forma rápida y moderna. Es el encargado de nuestro **backend**.

¿Qué es FastAPI?

Es un **framework de Python** para crear APIs web de forma rápida y moderna. Es el encargado de nuestro **backend**.

- Muy **rápido** (basado en estándares como ASGI y Uvicorn).
- **Documentación automática** e interactiva (Swagger UI en /docs).
- **Validación de datos** con *Pydantic* (tipos de Python).
- Define endpoints con decoradores como `@app.post("/train")`.

¿Qué es FastAPI?

Es un **framework de Python** para crear APIs web de forma rápida y moderna. Es el encargado de nuestro **backend**.

- Muy **rápido** (basado en estándares como ASGI y Uvicorn).
- **Documentación automática** e interactiva (Swagger UI en /docs).
- **Validación de datos** con *Pydantic* (tipos de Python).
- Define endpoints con decoradores como `@app.post("/train")`.

Con FastAPI recibimos el CSV, lo preprocesamos, entrenamos el modelo y devolvemos el resultado en JSON.

Streamlit: el frontend

¿Qué es Streamlit?

Es un **framework de Python** para construir **interfaces web** de datos y de machine learning **sin** escribir **HTML, CSS ni JavaScript**.

¿Qué es Streamlit?

Es un **framework de Python** para construir **interfaces web** de datos y de machine learning **sin escribir HTML, CSS ni JavaScript**.

- Todo se escribe en **Python** con funciones simples.
- Crea **widgets** en una línea: botones, selectores, subida de archivos.
- Ideal para **dashboards**, demos y prototipos de IA.
- Se ejecuta con: `streamlit run ui/app.py`.

Streamlit: el frontend

¿Qué es Streamlit?

Es un **framework de Python** para construir **interfaces web** de datos y de machine learning **sin escribir HTML, CSS ni JavaScript**.

- Todo se escribe en **Python** con funciones simples.
- Crea **widgets** en una línea: botones, selectores, subida de archivos.
- Ideal para **dashboards**, demos y prototipos de IA.
- Se ejecuta con: `streamlit run ui/app.py`.

En nuestra app

Streamlit muestra los botones y formularios, y cuando el usuario presiona “Entrenar” o “Predecir”, llama a FastAPI vía HTTP usando la librería `requests`.

Estructura del proyecto (carpetas)

Parte	Carpeta y contenido
Backend	app/ → servidor FastAPI (endpoints)
Frontend	ui/ → interfaz Streamlit
FastAPI	define /train y /predict/ en app/main.py
Streamlit	formularios y botones en ui/app.py
models/	Redes neuronales y entrenadores (TF y PyTorch)
utils/	Carga y preprocesamiento de datos
datasets/	Archivos CSV de datos
uploads/	Archivos subidos por el usuario

Estructura del proyecto (carpetas)

Parte	Carpeta y contenido
Backend	app/ → servidor FastAPI (endpoints)
Frontend	ui/ → interfaz Streamlit
FastAPI	define /train y /predict/ en app/main.py
Streamlit	formularios y botones en ui/app.py
models/	Redes neuronales y entrenadores (TF y PyTorch)
utils/	Carga y preprocesamiento de datos
datasets/	Archivos CSV de datos
uploads/	Archivos subidos por el usuario

Separation of concerns

Cada carpeta tiene una responsabilidad clara: es más fácil de mantener, probar y entender. Es una buena práctica de ingeniería de software.

Flujo de entrenamiento

- 1 El usuario sube un CSV en la interfaz (Streamlit).
- 2 Streamlit envía el archivo al backend (FastAPI) vía HTTP.
- 3 FastAPI guarda el archivo y lo preprocesa:
 - Separa X (características) y y (target).
 - Divide en entrenamiento (80 %) y prueba (20 %).
 - Normaliza los datos (StandardScaler).
- 4 Entrena la red (TensorFlow o PyTorch).
- 5 Guarda el modelo entrenado en disco.
- 6 Devuelve la ruta del modelo guardado.

¿Por qué normalizar?

Las variables tienen escalas muy diferentes (ej. $\text{radio} = 17$, $\text{área} = 1000$). Si una variable tiene números enormes, “domina” el aprendizaje.

¿Por qué normalizar?

Las variables tienen escalas muy diferentes (ej. radio = 17, área = 1000). Si una variable tiene números enormes, “domina” el aprendizaje.

StandardScaler

Transforma cada columna para que tenga **media 0** y **desviación 1**:

$$z = \frac{x - \text{media}}{\text{desviación}}$$

Así todas las variables tienen la misma importancia al empezar.

- 1 El usuario sube un CSV **sin** la columna target.
- 2 FastAPI preprocesa los datos (mismo proceso que en entrenamiento).
- 3 Carga el modelo ya entrenado.
- 4 El modelo genera una **probabilidad** para cada fila.
- 5 Se convierte a clase: $> 0,5 \rightarrow 1$ (benigno), si no $\rightarrow 0$ (maligno).
- 6 Se devuelve la lista de predicciones.

- 1 El usuario sube un CSV **sin** la columna target.
- 2 FastAPI preprocesa los datos (mismo proceso que en entrenamiento).
- 3 Carga el modelo ya entrenado.
- 4 El modelo genera una **probabilidad** para cada fila.
- 5 Se convierte a clase: $> 0,5 \rightarrow 1$ (benigno), si no $\rightarrow 0$ (maligno).
- 6 Se devuelve la lista de predicciones.

Importante

Los datos de predicción deben pasar por el **mismo** preprocesamiento que los de entrenamiento, o el modelo fallará.

El backend: FastAPI (endpoints)

```
@app.post("/train")
async def train_model(csv_file: UploadFile, framework: str = Form(...),
                      epochs: int = Form(20), target_column: str = Form(...)):
    # guarda el CSV, preprocesa y entrena
    X_train, y_train, X_test, y_test = prepare_tabular_data(temp_path,
                                                            target_column=target_column)

    if framework.lower() == "pytorch":
        model_path = trainer_pt.train_tabular(X_train, y_train, X_test, y_test, epochs)
    else:
        model_path = trainer_tf.train_tabular(X_train, y_train, X_test, y_test, epochs)
    return {"status": "ok", "saved_model": str(model_path)}
```

El backend: FastAPI (endpoints)

```
@app.post("/train")
async def train_model(csv_file: UploadFile, framework: str = Form(...),
                      epochs: int = Form(20), target_column: str = Form(...)):
    # guarda el CSV, preprocesa y entrena
    X_train, y_train, X_test, y_test = prepare_tabular_data(temp_path,
                                                            target_column=target_column)

    if framework.lower() == "pytorch":
        model_path = trainer_pt.train_tabular(X_train, y_train, X_test, y_test, epochs)
    else:
        model_path = trainer_tf.train_tabular(X_train, y_train, X_test, y_test, epochs)
    return {"status": "ok", "saved_model": str(model_path)}
```

- /train: entrena con el CSV subido.
- /predict/: predice sobre datos nuevos.
- Soporta data_type: tabular, imagen y audio.

La red en código (TensorFlow / Keras)

```
from tensorflow import keras
from tensorflow.keras import layers

def build_tabular_model(input_dim=30):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu"),
        layers.Dense(32, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ])
    model.compile(optimizer="adam",
                  loss="binary_crossentropy",
                  metrics=["accuracy"])
    return model
```

La red en código (TensorFlow / Keras)

```
from tensorflow import keras
from tensorflow.keras import layers

def build_tabular_model(input_dim=30):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu"),
        layers.Dense(32, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ])
    model.compile(optimizer="adam",
                  loss="binary_crossentropy",
                  metrics=["accuracy"])
    return model
```

Con solo unas líneas ya tenemos la red definida y lista para entrenar.

La red en código (PyTorch)

```
import torch.nn as nn

class TabularNet(nn.Module):
    def __init__(self, input_dim=30):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64), nn.ReLU(),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Linear(32, 16), nn.ReLU(),
            nn.Linear(16, 1), nn.Sigmoid(),
        )
    def forward(self, x):
        return self.net(x)
```

La red en código (PyTorch)

```
import torch.nn as nn

class TabularNet(nn.Module):
    def __init__(self, input_dim=30):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64), nn.ReLU(),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Linear(32, 16), nn.ReLU(),
            nn.Linear(16, 1), nn.Sigmoid(),
        )
    def forward(self, x):
        return self.net(x)
```

Misma arquitectura, pero en PyTorch escribimos la clase y el bucle de entrenamiento manualmente.

Pasos para ejecutar el proyecto

1 Crear y activar un entorno virtual:

```
python -m venv .venv  
.\.venv\Scripts\activate      # Windows  
source .venv/bin/activate     # Linux/Mac
```

Pasos para ejecutar el proyecto

1 Crear y activar un entorno virtual:

```
python -m venv .venv  
.\.venv\Scripts\activate      # Windows  
source .venv/bin/activate     # Linux/Mac
```

2 Instalar dependencias:

```
pip install -r requirements.txt
```

Pasos para ejecutar el proyecto

1 Crear y activar un entorno virtual:

```
python -m venv .venv
\\.venv\\Scripts\\activate      # Windows
source .venv/bin/activate     # Linux/Mac
```

2 Instalar dependencias:

```
pip install -r requirements.txt
```

3 Levantar el backend (terminal 1):

```
uvicorn app.main:app --reload --port 8000
```

Pasos para ejecutar el proyecto

1 Crear y activar un entorno virtual:

```
python -m venv .venv  
.\.venv\Scripts\activate      # Windows  
source .venv/bin/activate     # Linux/Mac
```

2 Instalar dependencias:

```
pip install -r requirements.txt
```

3 Levantar el backend (terminal 1):

```
uvicorn app.main:app --reload --port 8000
```

4 Levantar el frontend (terminal 2):

```
streamlit run ui/app.py
```

- Construimos una **red neuronal** que clasifica tumores de mama en benignos o malignos usando 30 características.
- Aprendimos el ciclo completo de un proyecto de IA: **datos** → **preprocesar** → **entrenar** → **predecir**.
- La misma arquitectura se implementó en **TensorFlow** y **PyTorch**, mostrando que el concepto es lo importante, no la herramienta.
- Separamos **backend** (FastAPI) y **frontend** (Streamlit), una buena práctica de arquitectura de software.

- Probar más arquitecturas y ajustar **hiperparámetros** (más capas, otras activaciones, learning rate).
- Añadir **métricas** extra: precisión, recall, F1 y matriz de confusión.
- Entrenar modelos de **imagen** y **audio** completos.
- Implementar **validación cruzada** para resultados más confiables.
- Desplegar la app en la nube (Docker + servidor).

¡Gracias!

¡Gracias por su atención!

Proyecto: NNapp
Red Neuronal para Detección de Cáncer de Mama

¿Preguntas?